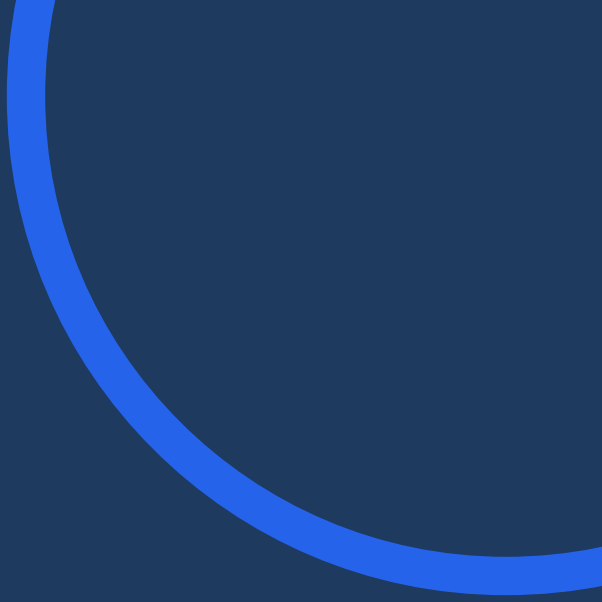

PHASE 3 · INTERMEDIATE · DAY 10 OF 30

Day 10 — Object-Oriented Programming I

Classes · Objects · __init__ · self · Instance & Class Variables

2 hours	5 topics	2 concept files	20 practice problems

30-Day Python Programming Course · Instructor Session Guide

Session Overview

Goal: Introduce students to Object-Oriented Programming by teaching them how to define classes, instantiate objects, use the `__init__` constructor, understand what `self` really means, and distinguish instance variables from class variables.

Outcome: By the end of this session, students will be able to design their own classes modelling real-world entities (bank accounts, shopping carts, tickets), create multiple objects from one blueprint, and correctly decide when a piece of data should live on the instance versus the class.

Session Timeline

Time	Segment	Focus
0:00 – 0:15	Why OOP?	Motivate classes vs. dictionaries; real-world analogies
0:15 – 0:45	Classes & __init__	Defining classes, instantiation, the constructor
0:45 – 1:05	Understanding self	What self is, how Python passes it automatically
1:05 – 1:15	Break	Short breather
1:15 – 1:45	Instance vs Class Vars	Shared vs. per-object data, attribute lookup rules
1:45 – 2:00	Instance Methods & Wrap-up	Practice problems, recap, Q&A

TIP

Students often confuse classes with the objects made from them. Keep returning to the “blueprint vs. house” analogy — a class is the blueprint; each object is a house built from it.

Prerequisites & Preparation

Requirements

Requirement	Notes
Completed Day 9	Error handling concepts referenced in examples
Python 3.12 environment active	Verify with python --version
VS Code open with day10/ folder	Concept + practice files ready to load
Comfort with functions (Day 5)	OOP methods build directly on function syntax

Instructor Pre-Session Checklist

- ■ Open day10_01_classes_basics.py and day10_02_instance_class_variables.py in VS Code
- ■ Run both files once beforehand to confirm clean output on this machine
- ■ Have the whiteboard/slide ready for the “blueprint vs. house” diagram
- ■ Prepare a simple real-world dictionary example (e.g. student record) for the opening contrast
- ■ Queue up day10_practise_problems_01 and _02 for the practice segment

Step-by-Step Instruction Guide

STEP 1 — Why Do We Need Classes?

What to Say:

"So far we've stored real-world things like students or tasks using dictionaries. That works, but it separates data from behaviour. Today we learn how to bundle both together using classes — the foundation of Object-Oriented Programming, or OOP."

```
student = {'name': 'Priya', 'marks': 88}
print(student['name'], 'scored', student['marks'])

# Output:
Priya scored 88
```

TIP

This dictionary has no behaviour of its own — any calculation (like a grade) needs a separate function elsewhere. Classes let us keep data and behaviour together.

STEP 2 — Defining a Class & __init__

What to Say:

"A class is a blueprint. On its own it doesn't do anything until we create objects from it. The __init__ method runs automatically every time we create a new object, letting us set up its starting data."

```
class Student:
    def __init__(self, name, city, marks):
        self.name = name
        self.city = city
        self.marks = marks

arjun = Student('Arjun', 'Delhi', 76)
print(arjun.name, arjun.city, arjun.marks)

# Output:
Arjun Delhi 76
```

Common Error	Fix
Forgetting self in method signature	Every instance method needs self as the first parameter

TypeError: missing positional argument	Check the number of arguments matches __init__'s signature
Using Student.name instead of arjun.name	Access instance attributes through the object, not the class

DONE WHEN

Students can each create a Student object for themselves (name, city, marks) and print a one-line summary without any errors.

STEP 3 — Understanding self

What to Say:

“self is just a reference to whichever object called the method. Python passes it in automatically — you never type it yourself when calling a method, only when defining one.”

```
class Task:
    def __init__(self, description, assigned_to):
        self.description = description
        self.assigned_to = assigned_to
        self.is_complete = False

    def mark_complete(self):
        self.is_complete = True

task1 = Task('Deliver groceries', 'Sanjay Verma')
task1.mark_complete()   # self = task1 here

# Output:
(is_complete becomes True on task1 only)
```

macOS/Linux: `python3 day10_01_classes_basics.py` | Windows: `python day10_01_classes_basics.py`

STEP 4 — Instance vs. Class Variables

What to Say:

“Instance variables belong to one object. Class variables are shared by every object made from that class — there's only ever one copy. If you assign to obj.attr, you always create an instance variable, even if a class variable with the same name exists.”

```
class Employee:
    company_name = 'TechNova Solutions'  # class variable
    employee_count = 0                   # class variable

    def __init__(self, name):
        self.name = name                 # instance variable
        Employee.employee_count += 1

meera = Employee('Meera')
arjun = Employee('Arjun')
print(Employee.employee_count)

# Output:
2
```

WATCH OUT

A common misconception is thinking self.employee_count += 1 updates the shared class variable — it actually creates a brand-new instance variable that shadows it. Always update class variables through the class name (Employee.employee_count).

STEP 5 — Instance Methods & Practice

What to Say:

“Now let's put it all together — a class with both instance and class variables, and instance methods that read and update them. Then we'll move into the practice problems.”

```
class ShoppingCart:
    tax_rate = 0.18   # class variable, 18% GST

    def __init__(self, customer_name):
        self.customer_name = customer_name
        self.subtotal = 0.0

    def total_with_tax(self):
        return self.subtotal * (1 + self.tax_rate)

cart = ShoppingCart('Riya Deshmukh')
cart.subtotal = 1148
print(round(cart.total_with_tax(), 2))

# Output:
1354.64
```

Direct students to day10_practise_problems_01_classes_basics.py and day10_practise_problems_02_instance_class_methods.py. Have them attempt Section B first, then C and D, saving Section E (Mixed Challenges) for anyone who finishes early.

Session Wrap-Up

Student Checklist

- ■ I can define a class with __init__ and create multiple objects from it
- ■ I understand that self refers to the specific object a method was called on
- ■ I can explain the difference between an instance variable and a class variable
- ■ I know that obj.attr = value always creates/overwrites an instance variable
- ■ I have completed at least Sections A–D of the practice problems

Key Concepts Recap

Concept	One-Line Summary
Class	A blueprint bundling data (attributes) and behaviour (methods)
Object / Instance	A concrete thing created from a class using ClassName()
__init__	Special method that auto-runs on object creation to set starting attributes
self	Reference to the specific object a method is being called on
Instance variable	Data unique to one object, set with self.attr = value
Class variable	Data shared by all objects, defined directly in the class body

Next Day Preview — Day 11: OOP II

Tomorrow builds directly on today's foundation: inheritance and super(), polymorphism, encapsulation (name mangling and properties), and dunder methods like __str__, __repr__, __len__, and __eq__.

WATCH OUT

If students are still shaky on self or the instance/class variable distinction, spend the first 10 minutes of Day 11 on a quick recap before introducing inheritance — it compounds fast.

Quick Reference Card

Daily Workflow Commands

```
cd day10
python day10_01_classes_basics.py
python day10_02_instance_class_variables.py
python day10_practise_problems_01_classes_basics.py
python day10_practise_problems_02_instance_class_methods.py
```

Syntax Cheat Sheet

```
class ClassName:
    class_var = value          # shared by all instances

    def __init__(self, param):
        self.instance_var = param   # unique per instance

    def some_method(self):
        return self.instance_var

obj = ClassName(value)         # instantiation
obj.some_method()              # calling an instance method
```

VS Code Shortcuts

Action	macOS	Windows/Linux
Run Python file	Ctrl+Option+N	Ctrl+Alt+N
Format document	Shift+Option+F	Shift+Alt+F
Toggle terminal	Ctrl+`	Ctrl+`
Go to definition	F12	F12

Tool Summary

File	Purpose
day10_01_classes_basics.py	Classes, objects, __init__, self
day10_02_instance_class_variables.py	Instance vs. class variables, instance methods
day10_practise_problems_01_classes_basics.py	Sections A–B practice problems

day10_practise_problems_02_instance_class_methods.py	Sections C–E practice problems